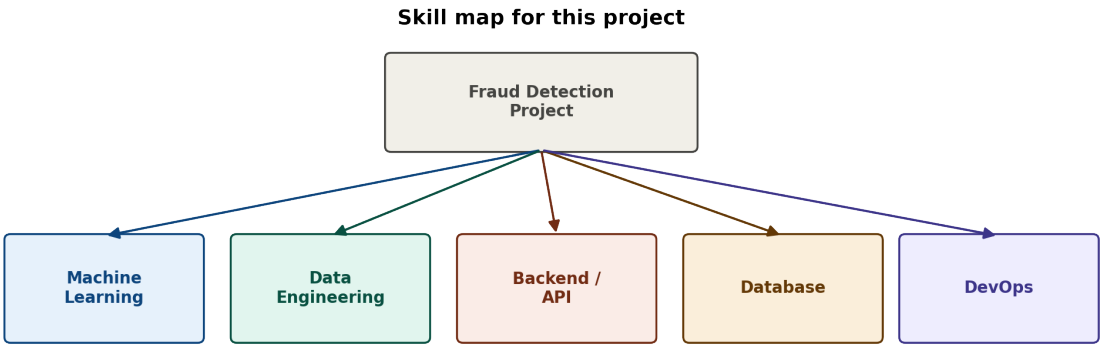
Skills Roadmap

Insurance Fraud Detection Project

What to study, topic by topic, to fully understand this project



1. How the project fits together

Before diving into individual topics, it helps to see the whole flow once. Raw claim data is cleaned, converted into numbers, used to train two models, evaluated, and finally served live through an API backed by a database.

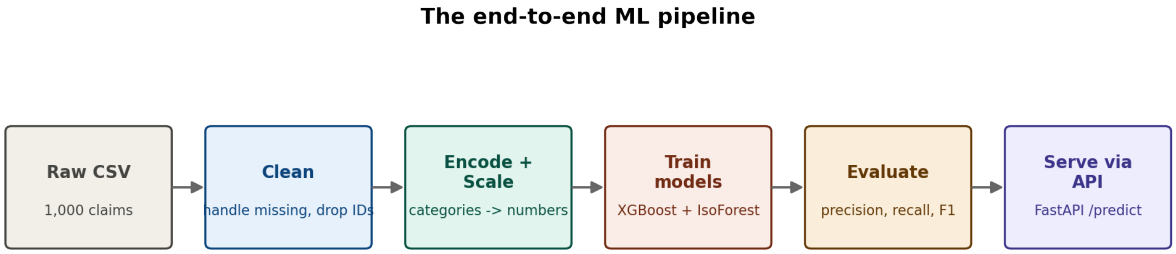


Figure 1 — the full pipeline, from raw CSV to a live prediction

---

2. Machine learning foundations

Core ideas you should be comfortable explaining before touching the code:

- What a **feature** and a **label** are, and the difference between them
- **Supervised vs unsupervised learning** — learning from labeled examples vs finding patterns with no labels
- **Classification** as a task: predicting a category (fraud / not fraud) rather than a number
- **Train/test split** and why you never evaluate a model on data it was trained on
- **Stratified sampling** — keeping the same class balance in both splits
- **Overfitting vs underfitting** — a model that memorizes training data vs one that's too simple to learn anything
- **Class imbalance** — why 75% accuracy can be a bad model when only 25% of examples are the class you actually care about

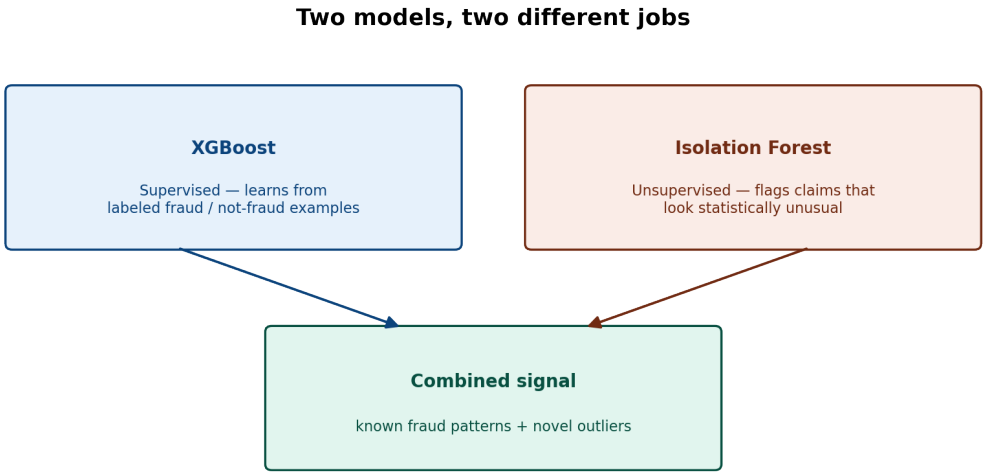


Figure 2 — why this project uses one of each kind of model

---

3. The two algorithms used

XGBoost (Gradient Boosting)

Study: decision trees, ensemble learning, and specifically **gradient boosting** — building trees one at a time where each new tree corrects the errors of all trees before it. Know what these hyperparameters control: *n_estimators* (number of trees), *max_depth* (how deep each tree grows), *learning_rate* (how much each tree corrects the last), and *subsample / colsample_bytree* (randomness used to fight overfitting).

Isolation Forest (Anomaly Detection)

Study: unsupervised anomaly detection and the core idea that outliers are "easy to isolate" with very few random splits, while normal points take many splits to separate from their neighbors. Know what *contamination* means (your prior belief about what fraction of data is anomalous).

---

4. Data preprocessing / feature engineering

- **Missing value imputation** — filling gaps with median (numeric) or most-frequent value (categorical)
- **One-hot encoding** — turning a category like "policy_state: OH" into binary columns
- **Feature scaling / standardization** — rescaling numbers so no single large-valued column dominates
- **sklearn Pipelines and ColumnTransformer** — chaining these steps into one reusable object
- **Data leakage** — why you must never let information from the test set influence training

---

5. Model evaluation

This is the single most important section to master — it's how you judge whether a model is actually good, not just "accurate."

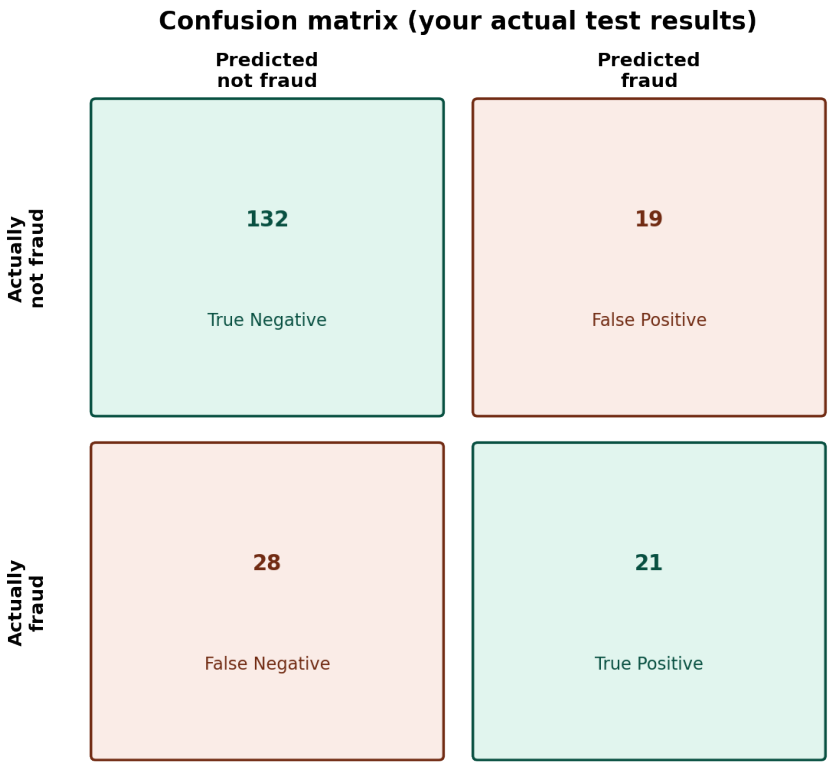


Figure 3 — the confusion matrix from this project's real test run

- **Confusion matrix** — true/false positives and negatives, the foundation of every other metric
- **Accuracy** — percent correct overall (misleading when classes are imbalanced)
- **Precision** — of everything flagged as fraud, how much really was fraud
- **Recall** — of all real fraud cases, how many did the model actually catch
- **F1-score** — a single number balancing precision and recall
- **ROC-AUC** — how well the model ranks fraud above non-fraud across every possible threshold
- **Decision threshold tuning** — the cutoff probability that turns a score into a yes/no label, and why 0.5 is rarely the best choice for imbalanced problems

---

6. Software engineering practices

- **Centralized configuration** — one file (config.py) holding every constant, so nothing is hardcoded twice
- **Separation of concerns** — preprocessing, training, evaluation, and serving live in separate modules
- **Serialization** — saving trained models to disk (joblib/pickle) so training and serving are separate steps
- **Caching** — loading expensive resources (like a trained model) only once, not on every request
- **Logging** — structured, timestamped logs instead of scattered print statements

---

7. Backend & API development

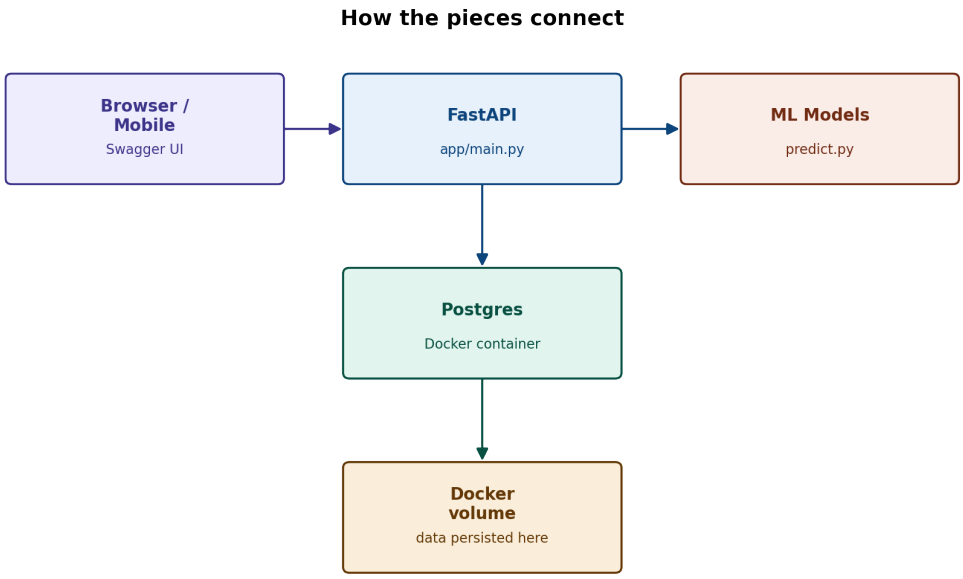


Figure 4 — request flow from client to model to database

- **REST API design** — resources, HTTP methods (GET/POST), status codes
- **FastAPI** — routing, path/query parameters, dependency injection (Depends)
- **Pydantic** — request/response validation, field aliasing, auto-generated schemas
- **Swagger / OpenAPI docs** — how an API can document itself automatically
- **Graceful degradation** — designing an API so one failing dependency (like a database) doesn't take down the whole service

---

8. Databases

- **Relational databases** and basic SQL concepts (tables, rows, columns, primary keys)
- **PostgreSQL** specifically — a production-grade open-source relational database
- **ORMs** (Object-Relational Mapping) — writing Python classes that map to database tables, as SQLAlchemy does here
- **Sessions and connection pooling** — why database connections are opened, used, and closed per-request

---

9. Infrastructure & DevOps

- **Containers** — what Docker actually is, and why it isolates an app from the host machine
- **Docker Compose** — running multiple containers (API + database) together as one system
- **Volumes** — how data persists independently of a container's lifecycle
- **Environment variables / .env files** — keeping configuration and secrets out of source code
- **Virtual environments** — isolating a project's Python packages from the rest of the system

10. Suggested study order

Stage	Focus	Where in the project
1	Python basics, pandas, numpy	preprocessing.py
2	Supervised learning + classification metrics	train.py, evaluate.py
3	Ensemble methods (XGBoost) & anomaly detection	train.py, config.py
4	REST APIs & FastAPI	app/main.py, app/schemas.py
5	Databases & SQLAlchemy	app/database.py
6	Docker & Docker Compose	Dockerfile, docker-compose.yml
7	Deployment (cloud hosting)	next: AWS / Render / Fly.io

Work top to bottom — each stage builds on the previous one. You already have stages 1 through 6 running and working on your machine. Stage 7 (deployment) is next.